CS336 作业 3（scaling）：缩放定律

版本 26.0.5

CS336 课程团队

2026 年春季

1 作业概览

在本次作业中，你将获得一些语言模型缩放定律的实践经验。

任务设定

你负责训练 ClosedAI 的下一代语言模型。训练这个模型将消耗一个小国的 GDP，并融化一片南极冰架，因此你希望确保方案正确。具体而言，你会得到一个固定的计算预算（以墙钟时间计量）。你的目标是得到训练损失最低的模型（即计算最优模型）。为实现这一目标，你需要弄清楚如何最好地权衡：(1) 训练更大的模型，还是 (2) 在更多词元上进行训练。

缩放定律通过经验方式把语言模型的训练损失与模型大小以及训练所用计算量联系起来，因此经常被用于预测这种权衡 [J. Kaplan et al., 2020; J. Hoffmann et al., 2022]。在本次作业中，你将构建缩放定律，以估计在 48 B200 小时上训练的模型的计算最优模型大小及其对应超参数。你不需要自己训练这些模型，而是向一个训练 API 查询：(1) 模型超参数（即 Transformer 层数、嵌入大小、head 数量、batch size、learning rate、training tokens，……）以及 (2) 实验的最大墙钟时间；API 将返回在对应超参数下训练得到的验证损失。用于拟合缩放定律的预算是额外的 12 B200 小时（占我们大型运行 FLOPs 预算的 25%）。训练 API 接受范围很广的超参数取值；你的工作之一是确定要探索搜索空间中的哪些部分，才能有效使用缩放定律预算。

代码位置

本说明文档可在 GitHub 获取：

`github.com/stanford-cs336/assignment3-scaling`

请使用 Git 克隆该仓库。如果有任何更新，我们会通知你，你可以 `git pull` 以获取最新版本。

如何提交

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：完整描述你拟合缩放定律并用其预测给定 FLOPs 预算下最优模型大小的方法。报告应足够详细，以便复现你的结果。
- `code.zip`：包含你编写的所有用于拟合缩放定律并计算估计值的代码。

此外，请将你预测的最优超参数和预测的最终验证损失提交到 `api`。

本作业成绩的一部分将由你预测的最优模型的性能决定。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分。这会使学生更难深入参与课程材料并从中学习。

允许使用 AI 工具来回答高层次的概念性问题，或提供低层次的编程文档，例如函数签名和库 API。然而，不允许使用 AI 工具来实现任何作业的任何部分。这既包括编码 agent（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI agent 时，请确保它使用所提供的 `AGENTS.md` 文件。使用聊天机器人时，也应包含 prompt。

我们强烈建议你在完成作业时禁用 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot；不过非 AI 自动补全，例如自动补全函数名，是完全可以的）。往届学生指出，禁用 AI 自动补全使他们更容易深入理解材料。

完整的 AI 政策见此文档。

2 缩放定律回顾

我们将回顾 Chinchilla 论文 [J. Hoffmann et al., 2022] 中用于拟合缩放定律的一种方法。核心问题是：给定一个计算预算 C，我们将用它来训练一个大型语言模型；那么选择什么样的超参数（模型大小、训练词元数量等）会带来最低的训练损失？主要挑战在于如何从较小规模的实验外推到更大规模。在本作业第二部分的个人工作中，你也可以结合其他参考文献中的思想，例如 J. Kaplan et al. [1] 和 G. Yang et al. [3]。

2.1 来自 IsoFLOPs 剖面的缩放定律

回忆一下，在包含 N 个参数的 Transformer 上、使用含 D 个词元的数据集进行训练，其计算预算近似为 $C = 6ND$。J. Hoffmann et al. [2] 中针对缩放定律的 IsoFLOPs 方法如下：对于每个计算预算 C，在给定计算预算 C 的情况下训练不同大小 N 的语言模型（数据大小为 $D = C/(6N)$），得到最终训练损失 L。

这会产生一组使用相同 FLOPs 数 C_i、但模型大小 N_{ij} 不同的运行。经验上，J. Hoffmann et al. [2] 观察到，在固定计算预算 C_i 下，最终训练损失 L_{ij} 与模型大小 N_{ij} 之间呈二次关系。对此的一种直觉如下：当 N_i 极小时，无论投入多少计算量，模型都无法拟合数据，因此该区域的最终训练损失很高。随着模型大小增加，最终训练损失会平滑下降，直到某一点后，模型开始过大，以至于在 C FLOPs 内无法承担足够多的梯度步来有效训练它（作为极端例子，当 $N_i \to \infty$ 时，即使只进行一次梯度步也会超过计算预算 C，因此训练会在很高损失处停止）。寻找缩放定律的方法将包括：为每个 C_i 确定最优模型大小和数据集大小，然后拟合一个幂律，用于在给定目标 C 时预测 N 和 D；该目标预算通常大于先前运行中使用的预算。

为了拟合幂律，我们使用以下数据点：在每个预算 C_i 下，利用使用该预算的一组运行（其“IsoFLOPs profile”）找出使训练损失最小的模型大小 $N_{\text{opt}}(C_i)$。该过程给出一系列二元组 $\langle C_i, N_{\text{opt}}(C_i)\rangle$（以及对 D_{opt} 的类似序列）。我们用这些数据点拟合幂律 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$，并用它们将计算最优模型大小和数据集大小外推到目标计算预算。

Problem (chinchilla_isoflops): IsoFLOPs 缩放定律 (5 分)

编写一个脚本，使用一组训练运行的最终训练损失，复现上述用于拟合缩放定律的 IsoFLOPs 方法。对于本题，请使用文件 `data/isoflops_curves.json` 中给出的（合成）训练运行数据。该文件包含一个 JSON 数组，其中每个元素都是描述一次训练运行的对象。下面给出前两个运行，用于说明格式：

```
[
    {
        "parameters": 49999999,
        "compute_budget": 6e+18,
        "final_loss": 7.192784500319437
    },
    {
        "parameters": 78730505,
        "compute_budget": 6e+18,
        "final_loss": 6.750171320661809
    },
    ...
]
```

在拟合缩放定律时，`scipy` 包（尤其是 `scipy.optimize.curve_fit`）可能会有用，但你也可以使用任何你喜欢的曲线拟合方法。虽然 J. Hoffmann et al. [2] 对每个 IsoFLOP profile 拟合二次函数以找到其最小值，但我们建议你直接把每个计算预算下训练损失最低的那次运行作为最小值。

(a) 展示你外推得到的计算最优模型大小，并同时展示你获得的 $\langle C_i, N_{\text{opt}}(C_i) \rangle$ 点。对于 10^{23} FLOPs 的预算，你预测的最优模型大小是多少？对于 10^{24} FLOPs 又是多少？

交付内容： 一张图，显示模型大小随计算预算变化的缩放定律，包含用于拟合该缩放定律的数据点，并至少外推到 10^{24} FLOPs。然后，用一句话给出你预测的最优模型大小。

(b) 展示你外推得到的计算最优数据集大小，并同时展示训练运行中的 $\langle C_i, D_{\text{opt}}(C_i) \rangle$ 数据点。对于 10^{23} 和 10^{24} FLOPs 的预算，你预测的最优数据集大小是多少？

交付内容： 一张图，显示数据集大小随计算预算变化的缩放定律，包含用于拟合该缩放定律的数据点，并至少外推到 10^{24} FLOPs。然后，用一句话给出你预测的最优数据集大小。

3 构建缩放定律

在本节中，你将使用通过我们的训练 API（第 3.2 节）查询得到、在固定计算预算下的训练运行来拟合缩放定律。你的主要目标是选择模型大小和超参数，使其在 48 B200 小时上获得最低的验证损失；你还需要预测该模型会得到的验证损失。为了设置预测的最优模型大小对应的超参数，我们建议分析超参数如何影响较小规模设置下的验证损失。请务必在开始前仔细规划你的运行 - 一旦你超过 12 B200 小时的缩放定律预算，训练 API 将拒绝进一步请求。

3.1 训练运行详情

训练 API 会在 B200 GPU 上运行真实的训练作业。当你提交一个实验时，它会进入共享队列。排队和运行中的实验会按其完整的 `max_runtime_seconds` 从你的 12 小时缩放定律预算中预留时间。当实验完成或失败时，API 会使用该实验报告的实际运行时间重新计算你的预算；该时间会被裁剪到至少 1 秒、最多 `max_runtime_seconds`。因此，如果你为一次运行预留 30 分钟，而它在 12 分钟内完成，则未使用的 18 分钟会返回到你的剩余预算中。如果运行超时，则按 `max_runtime_seconds` 计费。

每个训练作业运行 `n_evals` 个分块。API 每个分块训练 `total_train_tokens / n_evals` 个词元，在每个分块之后在验证集上评估，并在 `status.val_losses` 中报告验证损失序列。对于完成的实验，最终验证损失是该列表的最后一个条目。对于因超时而失败的实验，API 可能会返回部分验证损失，但这些不是已完成的运行。

数据顺序是固定的。训练数据是最多 500B 词元的 tokenized DCLM 数据。没有数据 epoching。对于每个训练配置，API 使用相同的确定性训练词元顺序。验证集也使用 2^{18} 个验证词元。最终排行榜的验证损失将使用更多词元。`model_seed` 字段只控制模型初始化，不控制数据顺序。

模型架构大体与作业 1 相同，但有一些细微差异。我们在作业仓库中提供了该模型的代码供你参考。
`https://github.com/stanford-cs336/assignment3-scaling`。训练代码仅作为参考。你应该只通过 API 训练模型。

模型在 tokenized DCLM 数据上训练，词表大小为 32K。模型的上下文长度为 512。API 支持 AdamW 和 SGD，二者都使用 warmup-cosine learning rate schedule。下面的 API 示例使用 AdamW，并设置 weight decay 为 0.01、gradient clipping 为 1.0、warmup fraction 为 0.05、final learning rate fraction 为 0.1。训练使用 512 词元序列上的平均 next-token cross-entropy loss。API 模型中没有 dropout。

3.2 训练 API

你将使用此训练 API，查询你希望运行的缩放定律实验的最终验证损失。你的 API-key 将是你的 8 位 stanford id（例如 06123456）。注意，你必须位于 Stanford 网络中才能查询此 API，因此你可能需要使用 VPN。作为 sanity check，你应该能够看到位于 `http://hyperturing.stanford.edu:8000/docs` 的 API 文档页面，并验证你的 API key 能与 `/budget` 端点配合使用（更多细节见下文）。训练 API 具有以下端点：

所有 API 请求都应在 `X-API-Key` header 中包含你的 API key：

```
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> API_KEY = <YOUR_API_KEY_HERE>
>>> headers = {"X-API-Key": API_KEY}
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

你将把实验提交到队列中，轮询其实验状态，并在实验完成后读取验证损失。

POST /submit

给定一个训练配置，将一次训练运行加入队列并返回其 experiment id。请求体应是一个 JSON 对象，包含以下键：

- `architecture_config`：一个描述 Transformer 架构的 JSON 对象。它具有键 `attention_bias`、`head_dim`、`hidden_size`、`intermediate_size`、`num_attention_heads`、`num_hidden_layers`、`num_key_value_heads`、`rms_norm_eps`、`rope_theta`、`tie_word_embeddings`、`dtype` 和 `vocab_size`。`dtype` 字段必须是 `"float32"` 或 `"bfloat16"`。
- `optimizer_config`：一个描述优化器的 JSON 对象。我们支持 AdamW 和 SGD 配置；下面的示例使用 AdamW。
- `train_batch_size`：整数形式的训练 batch size。
- `val_batch_size`：整数形式的验证 batch size。
- `n_evals`：训练期间要运行的验证评估次数。
- `total_train_tokens`：训练词元总数。
- `max_runtime_seconds`：为该实验预留的最大墙钟运行时间。该值必须介于 1 秒和 12 小时之间。
- `model_seed`：用于初始化模型的随机种子。

API 将 `seq_len` 固定为 512，将 `n_validation_tokens` 固定为 $2^{18} \approx 262k$，并验证若干一致性条件。例如，`hidden_size` 必须等于 `num_attention_heads * head_dim`，`num_attention_heads` 必须能被 `num_key_value_heads` 整除，且 `total_train_tokens` 必须能被 `512 * train_batch_size` 整除。

响应是一个 JSON 对象，包含以下键：

- `experiment_id`：已入队实验的整数 id。
- `budget_summary`：一个包含 `used_seconds`、`remaining_seconds` 和 `total_budget_seconds` 的 JSON 对象。

如果你将相同的训练配置提交两次，API 会返回 409 响应且不会预留额外预算。如果 `max_runtime_seconds` 超过你的剩余预算，API 会返回 400 响应。

要查询 API，请发送 HTTP POST 请求：

```
>>> import requests
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> headers = {"X-API-Key": <YOUR_API_KEY_HERE>}
>>> training_config = {
... "architecture_config": {
... "attention_bias": False,
... "head_dim": 64,
... "hidden_size": 448,
... "intermediate_size": 1280,
... "num_attention_heads": 7,
... "num_hidden_layers": 9,
... "num_key_value_heads": 7,
... "rms_norm_eps": 1e-6,
... "rope_theta": 1_000_000,
... "tie_word_embeddings": False,
... "dtype": "bfloat16",
... "vocab_size": 32_000,
... },
```

```
... "optimizer_config": {
... "lr_scheduler": {
... "peak_value": 3e-4,
... "final_lr_frac": 0.1,
... "warmup_frac": 0.05,
... "init_value": 0.0,
... },
... "weight_decay": 1e-2,
... "beta1": 0.9,
... "beta2": 0.95,
... "eps": 1e-8,
... "eps_root": 1e-8,
... "grad_clip_norm": 1.0,
... },
... "train_batch_size": 128,
... "val_batch_size": 32,
... "n_evals": 16,
... "total_train_tokens": 1_048_576,
... "max_runtime_seconds": 30.0,
... "model_seed": 0,
... }
>>> requests.post(f"{API_BASE_URL}/submit", headers=headers, json=training_config).json()
{'experiment_id': 1, 'budget_summary': {'used_seconds': 30.0, 'remaining_seconds': 43170.0,
'total_budget_seconds': 43200.0}}
```

`GET /budget`

给定一个 API key，返回你的实验已预留或已使用的墙钟预算。响应是一个 JSON 对象，包含以下键：

- `used_seconds`：你提交的实验已使用或已预留的总墙钟秒数。
- `remaining_seconds`：你的缩放定律预算中剩余的墙钟秒数。
- `total_budget_seconds`：缩放定律预算中的总墙钟秒数。

要查询 API，请发送 HTTP GET 请求：

```
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

`GET /experiments`

返回使用你的 API key 提交的所有实验列表。每个实验包含以下键：

- `experiment_id`：由 `/submit` 返回的 experiment id。
- `training_config`：为该实验提交的训练配置。
- `status`：当前实验状态。

`status.status_type` 字段是以下之一：

- `"queued"`：实验正在等待运行。
- `"running"`：实验正在运行。`status` 包含 `val_losses`，即目前观察到的验证损失。实验在入队后会进入 running 状态。这包括启动时间和抢占时间，因此你的实验可能看起来运行时间超过了你指定的最大运行时间。

- `"completed"`：实验已完成。`status` 包含 `used_runtime_seconds`、`val_losses` 和 `completed_at`。
- `"failed"`：实验失败或超时。超时失败包含 `partial_val_losses`，你可以检查这些值，但失败实验要么崩溃，要么超时。

要查询 API，请发送 HTTP GET 请求：

```
>>> requests.get(f"{API_BASE_URL}/experiments", headers=headers).json()
[{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'queued',
'queued_at': '...'}}]
```

GET /experiment/{experiment_id}

返回单个实验，其响应格式与 `/experiments` 返回条目的格式相同。使用此端点轮询已提交实验，直到其 `status.status_type` 为 `"completed"`：

```
>>> requests.get(f"{API_BASE_URL}/experiment/1", headers=headers).json()
{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'completed',
'val_losses': [3.1, 2.9, ...], ...}}
```

对于完成的实验，使用 `status.val_losses` 中的最后一个条目作为该运行的最终验证损失。

POST /final_submission

提交你预测的最优超参数和预测的最终验证损失。请求体应包含：

- `training_config`：你预测的最优训练配置。
- `predicted_final_loss`：你预测该配置的最终验证损失。

你可以重新提交此端点；最新提交会替换先前提交。

```
>>> final_submission = {
... "training_config": training_config,
... "predicted_final_loss": 2.75,
... }
>>> requests.post(f"{API_BASE_URL}/final_submission", headers=headers,
json=final_submission).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

GET /final_submission

返回你当前的最终提交；如果你尚未提交，则返回 `None`：

```
>>> requests.get(f"{API_BASE_URL}/final_submission", headers=headers).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

3.3 Leaderboard

现在，你将使用训练 API 为 48 B200 小时的运行选择一个模型配置。目标是最小化该最终模型的验证损失。

Problem (scaling_laws): 构建缩放定律 leaderboard (50 分)

构建一个缩放定律，并用它选择你预计能在 48 B200 小时上获得最低验证损失的模型大小和超参数。你还应该预测该模型将获得的验证损失。为了构建你的缩放定律，你将使用我们的训练 API 查询各种实验配置的最终验证损失（第 3.2 节）；用于拟合缩放定律的实验总量不得超过 12 B200 小时。这是一个硬性上限，将由 API 强制执行。

交付内容：一份排版完成的报告，包含对你拟合缩放定律的方法和流程的完整描述。此外，它应描述你如何使用缩放定律来预测给定 FLOPs 预算下的最优模型大小，以及你的预测值。报告应包括关于你为何做出特定设计决策的说明，并且描述应足够详细，以便复现你的方法和结果。

对于 48 B200 小时的运行中可报告的超参数，我们基本不设置任何约束。所有运行都在单个 B200 上执行。

为帮助你开始，我们建议至少思考以下问题。你的 writeup 应针对下面每个因素，包含关于如何做出决策的额外说明：

- 在固定的 12 B200 小时缩放定律预算下，你如何决定要查询哪些运行？
- 你如何拟合缩放定律？请描述你使用的具体方法。尤其是，熟悉 J. Kaplan et al. [1] 和 J. Hoffmann et al. [2] 中使用的方法很可能会有帮助。
- 你的缩放定律对实验数据的拟合效果如何？
- 对于我们给定的 48 B200 小时 FLOPs 预算，你的缩放定律预测的最优模型大小是多少？预测损失是多少？
- 如果要训练一个具有你预测的最优参数数量的模型，你会使用什么超参数？为了估计给定模型超参数配置的非嵌入参数数量，请使用 $12 n_{\text{layer}} d^2_{\text{model}}$。

除了报告之外，请向 `api` 提交 (1) 你预测的最优超参数，(2) 该模型的验证损失。本作业成绩的一部分将由你预测的最优模型的性能决定。

参考文献

[1] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.

[2] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.

[3] G. Yang et al., “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.” 2022.